

static operator()

Document #: P1169R4
Date: 2022-04-08
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Casey Carter
<casey@carter.net>

Contents

[1 Revision History](#)

[2 Motivation](#)

[3 Proposal](#)

[3.1 Overload Resolution](#)

[3.2 Lambdas](#)

[3.2.1 Static lambdas with capture](#)

[3.2.2 Can the static-ness of lambdas be implementation-defined?](#)

[3.3 Deduction Guides](#)

[3.4 Prior References](#)

[3.5 Implementation Experience](#)

[4 Wording](#)

[4.1 Language Wording](#)

[4.2 Library Wording](#)

[4.3 Feature-test macro](#)

[5 References](#)

§ 1 Revision History

Since [\[P1169R3\]](#), wording.

Since [\[P1169R2\]](#), added missing feature-test macro and updated wording to include [\[LWG3617\]](#).

[\[P1169R1\]](#) was approved for electronic polling by EWG, but two issues came up that while this paper does not *change* are still worth commenting on: can [static lambdas still have capture](#) and can whether or not stateless lambdas be `static` be [implementation-defined](#)?

[\[P1169R0\]](#) was presented to EWGI in San Diego, where there was no consensus to pursue the paper. However, recent discussion has caused renewed interest in this paper so it has been resurfaced. R0 of this paper additionally proposed implicitly changing capture-less lambdas to have static function call

operators, which would be a breaking change. That part of this paper has been changed to instead allow for an explicit opt-in to static. Additionally, this language change has been implemented.

§ 2 Motivation

The standard library has always accepted arbitrary function objects - whether to be unary or binary predicates, or perform arbitrary operations. Function objects with call operator templates in particular have a significant advantage today over using overload sets since you can just pass them into algorithms. This makes, for instance, `std::less<>{}` very useful.

As part of the Ranges work, more and more function objects are being added to the standard library - the set of Customization Point Objects (CPOs). These objects are Callable, but they don't, as a rule, have any members. They simply exist to do what Eric Niebler termed the “[Std Swap Two-Step](#)”. Nevertheless, the call operators of all of these types are non-static member functions. Because *all* call operators have to be non-static member functions.

What this means is that if the call operator happens to not be inlined, an extra register must be used to pass in the `this` pointer to the object - even if there is no need for it whatsoever. Here is a [simple example](#):

```
struct X {
    bool operator()(int) const;
    static bool f(int);
};

inline constexpr X x;

int count_x(std::vector<int> const& xs) {
    return std::count_if(xs.begin(), xs.end(),
#ifdef STATIC
        X::f
    #else
        x
    #endif
    );
}
```

`x` is a global function object that has no members that is intended to be passed into various algorithms. But in order to work in algorithms, it needs to have a call operator - which must be non-static. You can see the difference in the generated asm between using the function object as intended and passing in an equivalent static member function:

Non-static call operator	Static member function
<pre>count_x(std::vector<int, std::allocator<int> > const&): push r12 push rbp push rbx sub rsp, 16</pre>	<pre>count_x(std::vector<int, std::allocator<int> > const&): push r12 push rbp push rbx mov r12, QWORD PTR [rdi+8]</pre>

Non-static call operator		Static member function	
	<pre> mov r12, QWORD PTR [rdi+8] mov rbx, QWORD PTR [rdi] mov BYTE PTR [rsp+15], 0 cmp r12, rbx je .L5 xor ebp, ebp .L4: mov esi, DWORD PTR [rbx] lea rdi, [rsp+15] call X::operator()(int) const cmp al, 1 sbb rbp, -1 add rbx, 4 cmp r12, rbx jne .L4 add rsp, 16 mov eax, ebp pop rbx pop rbp pop r12 ret .L5: add rsp, 16 xor eax, eax pop rbx pop rbp pop r12 ret </pre>		<pre> mov rbx, QWORD PTR [rdi] cmp r12, rbx je .L5 xor ebp, ebp .L4: mov edi, DWORD PTR [rbx] call X::f(int) cmp al, 1 sbb rbp, -1 add rbx, 4 cmp r12, rbx jne .L4 mov eax, ebp pop rbx pop rbp pop r12 ret .L5: pop rbx xor eax, eax pop rbp pop r12 ret </pre>

Even in this simple example, you can see the extra zeroing out of `[rsp+15]`, the extra `lea` to move that zero-ed out area as the object parameter - which we know doesn't need to be used. This is wasteful, and seems to violate the fundamental philosophy that we don't pay for what we don't need.

The typical way to express the idea that we don't need an object parameter is to declare functions `static`. We just don't have that ability in this case.

§ 3 Proposal

The proposal is to just allow the ability to make the call operator a static member function, instead of requiring it to be a non-static member function. We have many years of experience with member-less function objects being useful. Let's remove the unnecessary object parameter overhead. There does not seem to be any value provided by this restriction.

There are other operators that are currently required to be implemented as non-static member functions - all the unary operators, assignment, subscripting, conversion functions, and class member access. We do not believe that being able to declare any of these as static will have as much value, so we are not pursuing those at this time. We're not aware of any use-case for making any of these other operators static, while the use-case of having stateless function objects is extremely common.

§ 3.1 Overload Resolution

There is one case that needs to be specially considered when it comes to overload resolution, which did not need to be considered until now:

```
struct less {
    static constexpr auto operator()(int i, int j) -> bool {
        return i < j;
    }

    using P = bool (*)(int, int);
    operator P() const { return operator(); }
};

static_assert(less{}(1, 2));
```

If we simply allow `operator()` to be declared `static`, we'd have two candidates here: the function call operator and the surrogate call function. Overload resolution between those candidates would work as considering between:

```
operator()(contrived-parameter, int, int);
call-function(bool (*)(int, int), int, int);
```

And currently this is ambiguous because 12.2.4.1 [\[over.match.best.general\]/1.1](#) stipulates that the conversion sequence for the contrived implicit object parameter of a static member function is neither better nor worse than *any other conversion sequence*. This needs to be reined in slightly such that the conversion sequence for the contrived implicit object parameter is neither better nor worse than any *standard* conversion sequence, but still better than user-defined or ellipsis conversion sequences. Such a change would disambiguate this case in favor of the call operator.

§ 3.2 Lambdas

A common source of function objects whose call operators could be static but are not are lambdas without any capture. Had we been able to declare the call operator static when lambdas were originally introduced in the language, we would surely have had a lambda such as:

```
auto four = []{ return 4; };
```

desugar into:

```
struct __unique {
    static constexpr auto operator()() { return 4; };

    using P = int();
    constexpr operator P*() { return operator(); }
};

__unique four{};
```

Rather than desugaring to a type that has a non-static call operator along with a conversion function that has to return some other function.

However, we can't simply change such lambdas because this could break code. There exists code that takes a template parameter of callable type and does `decltype(&F::operator())`, expecting the resulting type to be a pointer to member type (which is the only thing it can be right now). If we change captureless lambdas to have a static call operator implicitly, all such code would break for captureless lambdas. Additionally, this would be a language ABI break. While lambdas shouldn't show up in your ABI anyway, we can't with confidence state that such code doesn't exist nor that such code deserves to be broken.

Instead, we propose that this can be opt-in: a lambda is allowed to be declared `static`, which will then cause the call operator (or call operator template) of the lambda to be a static member function rather than a non-static member function:

```
auto four = []() static { return 4; };
```

We then also need to ensure that a lambda cannot be declared `static` if it is declared `mutable` (an inherently non-static property) or has any capture (as that would be fairly pointless, since you could not access any of that capture).

§ 3.2.1 Static lambdas with capture

Consider the situation where a lambda may need to capture something (for lifetime purposes only) but does not otherwise need to reference it. For instance:

```
auto under_lock = [lock=std::unique_lock(mtx)]() static { /* do something */; };
```

The body of this lambda does not use the capture `lock` in any way, so there isn't anything that inherently prevents this lambda from having a `static` call operator. The rule from R1 of this paper was basically:

A `static` lambda shall have no *lambda-capture*.

But could instead be:

If a lambda is `static`, then any *id-expression* within the body of the lambda that would be an odr-use of a captured entity is ill-formed.

However, we feel that the value of the teachability of “Just make stateless lambdas `static`” outweighs the value of supporting holding capturing variables that the body of the lambda does not use. This restriction could be relaxed in the future, if it proves overly onerous (much as we are here relaxing the restriction that call operators be non-static member functions).

This aspect was specifically polled during the telecon, and the outcome was:

SF	F	N	A	SA
0	3	4	5	0

§ 3.2.2 Can the `static`-ness of lambdas be implementation-defined?

Another question arose during the telecon about whether it is feasible or desirable to make it implementation-defined as to whether or not the call operator of a capture-less lambda is `static`.

The advantage of making it implementation-defined is that implementations could, potentially, add a flag that would allow users to treat all of their capture-less lambdas as `static` without the burden of adding this extra annotation (had call operators been allowed to be static before C++11, surely capture-less lambdas would have been implicitly `static`) while still making this sufficiently opt-in as to avoid ABI-breaking changes.

The disadvantage of making it implementation-defined is that this is a fairly important property of how a lambda behaves. Right now, the observable properties of a lambda are specified and portable. The implementation freedom areas are typically not observable to the programmer. The static-ness of the operator is observable, so making that implementation-defined or unspecified seems antithetical to the design of lambdas. The rationale for doing something like this (i.e. avoiding a sea of seemingly-pointless `static` annotations when the compiler should be able to Just Do It), but it seems rather weird that a property like that wouldn't be portable.

§ 3.3 Deduction Guides

Consider the following, assuming a version of `less` that uses a static call operator:

```
template <typename T>
struct less {
    static constexpr auto operator()(T const& x, T const& y) -> bool {
        return x < y;
    };
};

std::function f = less<int>{};
```

This will not compile with this change, because `std::function`'s deduction guides only work with either function pointers (which does not apply) or class types whose call operator is a non-static member function. These will need to be extended to support call operators with function type (as they would for [P0847R6](#) anyway).

§ 3.4 Prior References

This idea was previously referenced in [EWG88](#), which reads:

In c++std-core-14770, Dos Reis suggests that `operator[]` and `operator()` should both be allowed to be static. In addition to that, he suggests that both should allow multiple parameters. It's well known that there's a possibility that this breaks existing code (`foo[1,2]` is valid, the thing in brackets is a comma-expression) but there are possibilities to fix such cases (by requiring parens if a comma-expression is desired). EWG should discuss whether such unification is to be strived for.

Discussed in Rapperswil 2014. EWG points out that there are more issues to consider here, in terms of other operators, motivations, connections with captureless lambdas, who knows what

else, so an analysis paper is requested.

There is a separate paper proposing multi-argument subscripting [P2128R3] already, with preexisting code such as `foo[1, 2]` already having been deprecated.

§ 3.5 Implementation Experience

The language changes have been implemented in EDG.

§ 4 Wording

§ 4.1 Language Wording

Add `static` to the grammar of 7.5.5.1 [expr.prim.lambda.general]:

```
lambda-specifier:  
  consteval  
  constexpr  
  mutable  
+ static
```

Change 7.5.5.1 [expr.prim.lambda.general]/4:

- 4 A *lambda-specifier-seq* shall contain at most one of each *lambda-specifier* and shall not contain both `constexpr` and `consteval`. If the *lambda-declarator* contains an explicit object parameter ([dcl.fct]), then no *lambda-specifier* in the *lambda-specifier-seq* shall be `mutable` or `static`. The *lambda-specifier-seq* shall not contain both `mutable` and `static`. If the *lambda-specifier-seq* contains `static`, there shall be no *lambda-capture*.

Change 7.5.5.2 [expr.prim.lambda.closure]/5:

- 5 The function call operator or operator template is a `static member function` or `static member function template` ([class.static.mfct]) if the *lambda-expression's parameter-declaration-clause* is followed by `static`. Otherwise, it is a `non-static member function` or `member function template` ([class.mfct.non-static]) that is declared `const` ([class.mfct.non.static]) if and only if the *lambda-expression's parameter-declaration-clause* is not followed by `mutable` and the *lambda-declarator* does not contain an explicit object parameter. It is neither virtual nor declared `volatile`. Any *noexcept-specifier* specified on a *lambda-expression* applies to the corresponding function call operator or operator template. An *attribute-specifier-seq* in a *lambda-declarator* appertains to the type of the corresponding function call operator or operator template. The function call operator or any given operator template specialization is a `constexpr` function if either the corresponding *lambda-expression's parameter-declaration-clause* is followed by `constexpr`, or it satisfies the requirements for a `constexpr` function.

Add a note to 7.5.5.2 [expr.prim.lambda.closure]/8 and /11 indicating that we could just return the call operator. The wording as-is specifies the behavior of the return here, and returning the call operator

already would be allowed, so no wording change is necessary. But the note would be helpful:

- 8 The closure type for a non-generic *lambda-expression* with no *lambda-capture* whose constraints (if any) are satisfied has a conversion function to pointer to function with C++ language linkage having the same parameter and return types as the closure type's function call operator. The conversion is to "pointer to `noexcept` function" if the function call operator has a non-throwing exception specification. **If the function call operator is a static member function, then the value returned by this conversion function is the address of the function call operator. Otherwise, the ~~The~~ value returned by this conversion function is the address of a function F that, when invoked, has the same effect as invoking the closure type's function call operator on a default-constructed instance of the closure type. F is a constexpr function if the function call operator is a constexpr function and is an immediate function if the function call operator is an immediate function.**
- 11 **If the function call operator template is a static member function template, then the value returned by any given specialization of this conversion function template is the address of the corresponding function call operator template specialization. Otherwise, the ~~The~~ value returned by any given specialization of this conversion function template is the address of a function F that, when invoked, has the same effect as invoking the generic lambda's corresponding function call operator template specialization on a default-constructed instance of the closure type. F is a constexpr function if the corresponding specialization is a constexpr function and F is an immediate function if the function call operator template specialization is an immediate function.**

Change 12.2.4.1 [\[over.match.best.general\]](#)/1 to drop the static member exception and remove the bullets and the footnote:

- 1 Define $ICS^i(F)$ as ~~follows:~~
- (1.1) — ~~If E is a static member function, $ICS^1(E)$ is defined such that $ICS^1(E)$ is neither better nor worse than $ICS^1(G)$ for any function G, and, symmetrically, $ICS^1(G)$ is neither better nor worse than $ICS^1(E)$; ¹¹⁷ otherwise,~~
- (1.2) — ~~let $ICS^i(E)$ denote~~ the implicit conversion sequence that converts the i^{th} argument in the list to the type of the i^{th} parameter of viable function F. [\[over.best.ics\]](#) defines the implicit conversion sequences and [\[over.ics.rank\]](#) defines what it means for one implicit conversion sequence to be a better conversion sequence or worse conversion sequence than another.

Add to 12.2.4.2.1 [\[over.best.ics.general\]](#) a way to compare this static member function case:

- * **When the parameter is the implicit object parameter of a static member function, the implicit conversion sequence is a standard conversion sequence that is neither better nor worse than any other standard conversion sequence.**

Change 12.4 [\[over.oper\]](#) paragraph 6 and introduce bullets to clarify the parsing. `static void operator()() { }` is a valid function call operator that has no parameters with this proposal, so needs to

be clear that the “has at least one parameter” part refers to the non-member function part of the clause.

6 An operator function shall either

- (6.1) — be a ~~non-static~~ member function or
- (6.2) — be a non-member function that has at least one parameter whose type is a class, a reference to a class, an enumeration, or a reference to an enumeration.

It is not possible to change the precedence, grouping, or number of operands of operators. The meaning of the operators `=`, (unary) `&`, and `,` (comma), predefined for each type, can be changed for specific class and enumeration types by defining operator functions that implement these operators. Operator functions are inherited in the same manner as other base class functions.

Change 12.4.4 [\[over.call\]](#) paragraph 1:

- 1 A *function call operator function* is a function named `operator()` that is a ~~non-static~~ member function with an arbitrary number of parameters.

§ 4.2 Library Wording

Change the deduction guide for function in 20.14.17.3.2 [\[func.wrap.func.con\]](#)/16-17. [Editor's note: This assumes the wording change in [\[LWG3617\]](#). This relies on the fact that `f.operator()` would be valid for a static member function, but not an explicit object member function - which like other non-static member functions you can't just write `x.f` you can only write `x.f(args...)`.]:

```
template <class F> function(F) -> function<see below>;
```

- 15 *Constraints:* `&F::operator()` is well-formed when treated as an unevaluated operand and either

- (15.1) — `F::operator()` is a non-static member function and `decltype(&F::operator())` is either of the form `R(G::*)(A...) cv &_opt noexcept_opt` or of the form `R(*) (G cv ref_opt, A...) noexcept_opt` for a type `G`, or
- (15.2) — `F::operator()` is a static member function and `decltype(&F::operator())` is of the form `R(*) (A...) noexcept_opt`.

- 16 *Remarks:* The deduced type is `function<R(A...)>`.

Change the deduction guide for `packaged_task` in 32.9.10.2 [\[futures.task.members\]](#)/7-8 in the same way:

```
template <class F> packaged_task(F) -> packaged_task<see below>;
```

- 7 *Constraints:* `&F::operator()` is well-formed when treated as an unevaluated operand and either

- (7.1) — `F::operator()` is a non-static member function and `decltype(&F::operator())` is either of the form `R(G::*)(A...) cv &_opt noexcept_opt` or of the form `R(*) (G cv`

`refopt, A...`) noexcept_{opt} for a type `G`, or

(7.2) — `F::operator()` is a static member function and `decltype(&F::operator())` is of the form `R(*) (A...) noexceptopt`.

8 *Remarks:* The deduced type is `packaged_task<R(A...)>`.

§ 4.3 Feature-test macro

Add to 15.11 [\[cpp.predefined\]](#)/table 19:

`__cpp_static_call_operator`

with the appropriate value. This allows define function objects or lambdas to have conditionally static call operators when possible.

§ 5 References

[EWG88] Gabriel Dos Reis. [tiny] Uniform handling of `operator[]` and `operator()`.

<https://wg21.link/ewg88>

[LWG3617] Barry Revzin. 2021. `function/packaged_task` deduction guides and deducing `this`.

<https://cplusplus.github.io/LWG/issue3617>

[P0847R6] Barry Revzin, Gašper Ažman, Sy Brand, Ben Deane. 2021-01-15. Deducing `this`.

<https://wg21.link/p0847r6>

[P1169R0] Barry Revzin, Casey Carter. 2018-10-07. `static operator()`.

<https://wg21.link/p1169r0>

[P1169R1] Barry Revzin, Casey Carter. 2021-04-06. `static operator()`.

<https://wg21.link/p1169r1>

[P1169R2] Barry Revzin, Casey Carter. 2021-08-14. `static operator()`.

<https://wg21.link/p1169r2>

[P1169R3] Barry Revzin, Casey Carter. 2021-10-14. `static operator()`.

<https://wg21.link/p1169r3>

[P2128R3] Corentin Jabot, Isabella Muerte, Daisy Hollman, Christian Trott, Mark Hoemmen. 2021-02-15. Multidimensional subscript operator.

<https://wg21.link/p2128r3>